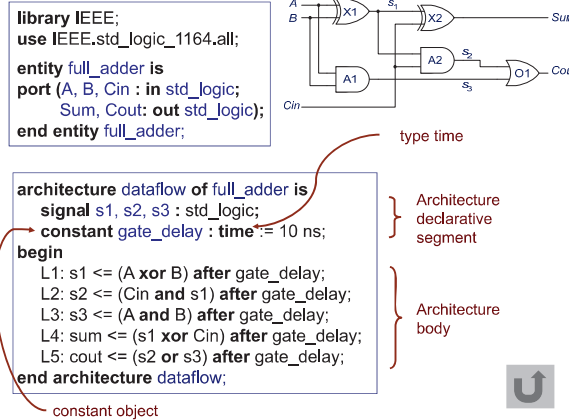


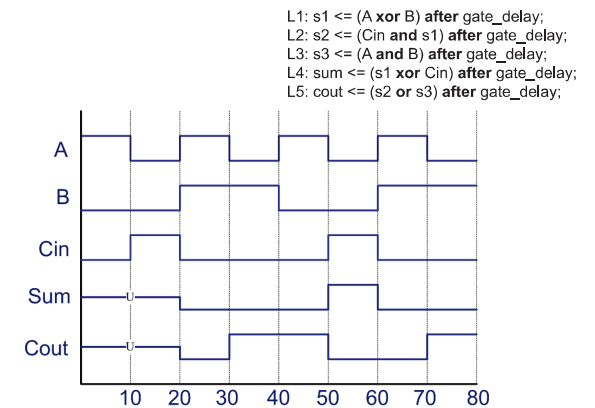
EE332 Digital System Design

Dr. Yi Zeng
Assistant Professor,
Department of Electrical and Electronic Engineering
Office: S236, South Tower of COE
Email: zengyi@sustech.edu.cn

1



78



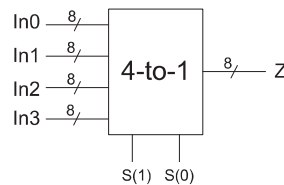
79

3.2 Conditional signal assignment

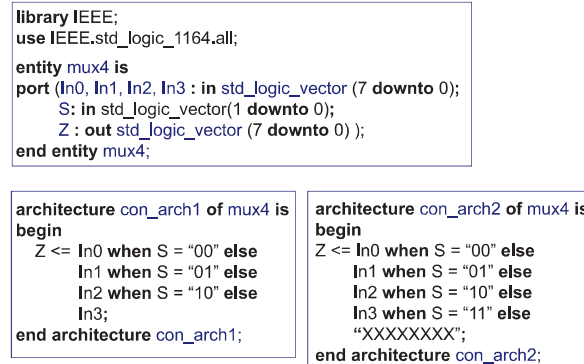
- Simple CSAs is convenient for describing gate-level circuits whose behavior can be expressed with Boolean equations.
- It is useful to model circuits at higher levels of abstraction such as multiplexors, decoders, priority encoders, etc.

4-to-1, 8-bit multiplexor

Input S	Output Z
00	In0
01	In1
10	In2
11	In3



80



81

- The conditional signal assignment itself is a concurrent signal assignment.
- It has one target, but can have more than one expression.
General form:

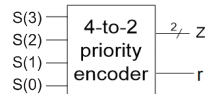
```

target <= {expression} when {condition} else
          {expression} when {condition} else
          .....
          {expression};

```
- When this CSA is executed, the expressions in the right hand side are evaluated in the order that they appear.
 - The order of the expressions with their respective conditions inside the statement is important.
- Even there are several lines of text, this corresponds to only one signal assignment statement.

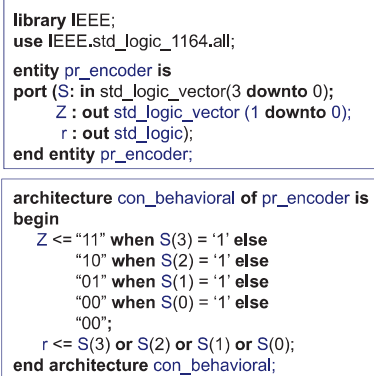
82

4-to-2 priority encoder



Assume that S(3) has the highest priority

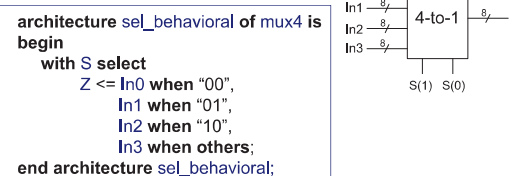
Input S	Output Z, r
1 ---	11, 1
0 1 --	10, 1
0 0 1 -	01, 1
0 0 0 1	00, 1
0 0 0 0	00, 0



83

3.3 Selected signal assignment

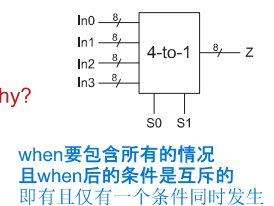
- It is similar to the conditional signal assignment statement.
- The value of a target signal is determined by the value of a select expression.
- Example: the same 4-1 multiplexer.



87

with S select
 Z <= In0 when "00",
 In1 when "01",
 In2 when "10",
 In3 when "11";

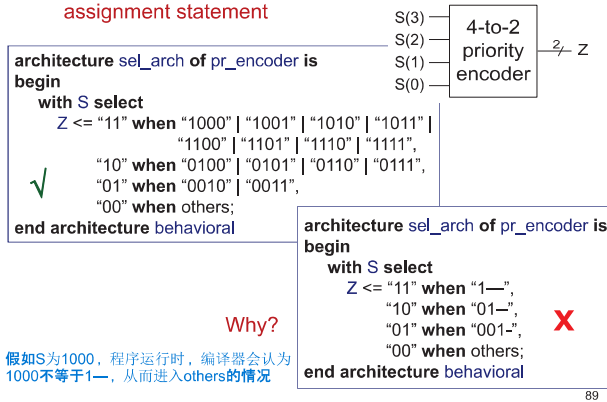
Why?



with S select
 Z <= In0 when "00",
 In1 when "01",
 In2 when "10",
 In3 when "11",
 "XXXXXXXX" when others;

88

4-to-2 priority encoder based on a selected signal assignment statement



89

- The choices for the select expression are not evaluated in sequence.
- All choices are evaluated, but one and only one must be true.
- All of the choices that the programmer specifies must cover all the possible values of the select expression.
- When an event occurs on a signal used in the select expression, or any of the signals used in one of the choices, the statement is executed.

90

3.4 Understanding delays

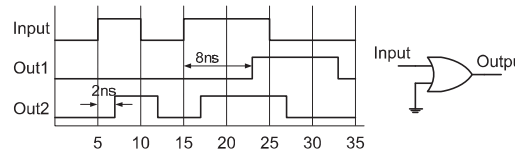
- Accurate representation of the behavior of digital circuits requires accurate modeling of delays through the various components.
- Three types of delay models
 - Inertial delay model
 - Transport delay model
 - Delta delay

93

3.4.1 The inertial delay model

- Digital circuits takes a finite amount of time for the output of a gate to respond to a change on the input.
- This implies that the change on the input has to persist for a certain period of time to ensure that the output will response.
- If it does not persist long enough the input events will not be propagated to the output.
- This propagation delay model is referred to as the inertial delay model.

94



Out1 <= Input or '0' after 8 ns;
Out2 <= Input or '0' after 2 ns;

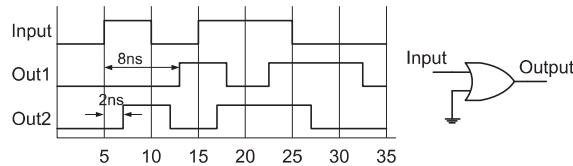
- Any pulse in the input with a width of less than the propagation delay through the gate is said to be rejected.
- often used for component delays
- Default in VHDL program
默认为惯性delay

95

3.4.2 The transport delay model

- Transport delay models the delays in hardware that do not exhibit any inertial delay.
- This delay represents pure propagation delay; that is, any changes on an input are transported to the output, no matter how small of the width, after the specified delay.
- Keyword **transport** is used in a signal assignment statement for transport delay model.

96



Out1 <= transport (Input or '0') after 8 ns;
Out2 <= transport (Input or '0') after 2 ns;

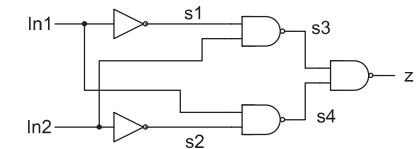
- Pulses are propagated, irrespective of width
- good for interconnect delays.

97

3.4.3 Delta Delays

- If we do not specify a delay for the occurrence of an event on a signal, for example
sum <= (x xor y);
a delta delay is assumed by the simulator
- Delta delay is an infinitesimally small delay, Δ.
- In a signal assignment, the value is not assigned to the signal directly but after a delta delay at the earliest.
- Delta delays are simply used to enforce dependencies between events and thereby ensure correct simulation.

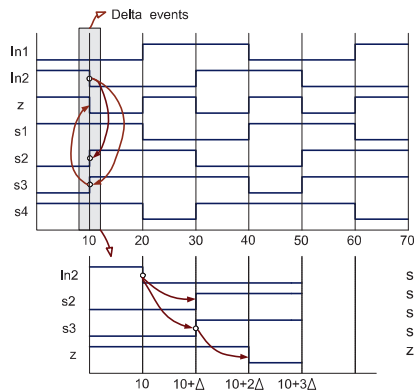
98



library IEEE;
use IEEE.std_logic_1164.all;
entity combinational is
port (In1, In2 : in std_logic;
z : out std_logic);
end entity combinational;

architecture behavior of combinational is
signal s1, s2, s3, s4 : std_logic := 0;
begin
s1 <= not In1;
s2 <= not In2;
s3 <= not (s1 and In2);
s4 <= not (s2 and In1);
z <= not (s3 and s4);
end architecture behavior;

99



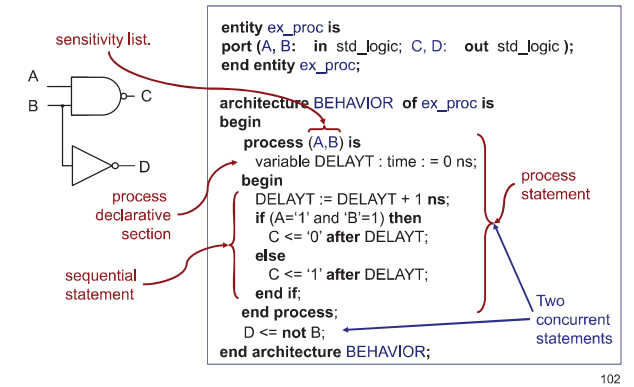
100

Chapter 4. Sequential VHDL

- Sequential statements are specified inside a process.
- Processes represent fundamental method by which concurrent activities are modeled.

不同的process之间是并行的，每一个process中是顺序执行的

4.1. The process construct



102

- A process is a large concurrent statement that itself contains a series of sequential statements.
- The body of the process is treated by VHDL as a single concurrent statement and is executed at the same time as all other concurrent statements in the simulation.

103

Process with a sensitivity list

- The sensitivity list is a list of signal to which the process responds.

```
signal a, b, c, y: std_logic; -- in architecture declaration
...
process (a, b, c) -- in architecture body
begin
    y <= a and b and c; -- infer a combinational circuit
end process;
```

```
signal a, b, c, y: std_logic;
...
process (a) -- a process with incomplete sensitivity list
-- not infer a combinational circuit
begin
    y <= a and b and c;
end process;
```

当sensitivity list中的信号发生变化时，对应process将会被执行

104

Process with a wait statement

- A process with wait statements has one or more wait statement but no sensitivity list.

wait on signals;
wait until boolean_expression;
wait for time_expression;

```
process
begin
    y <= a and b and c;
    wait on a, b, c;
end process;
```

In synthesis, only few well defined forms of wait statement can be used, and normally only one wait statement is allowed in a process.
Not recommended.

105

```
process (a, b) is
begin
    c <= a and b;
end process;
```

sensitivity list. The process will be activated if either a or b (or both) changes states

```
Process is
begin
    c <= a and b;
    wait on a, b;
end process;
```

A process can have wait statement or sensitivity list, but not both

```
process is
begin
    c <= a and b;
end process;
```

This is an infinite loop.

106

4.2. Sequential statements

- sequential signal assignment statement
- variable assignment statement
- if statement
- case statement
- null statement
- wait statement
- loop statement
- exit statement
- next statement

107

4.2.1 Signal assignment statement

Signal-object <= expression [after delay-value];

- Outside a process – concurrent signal assignment statement
- Within a process – sequential signal assignment, executed in sequence with respect to the other sequential statement which appear within that process.
- When a signal assignment statement is executed, the value of expression is computed immediately, and this computed value is scheduled to be assigned to the signal after the specified delay.

108

- Inside a process, a signal can be assigned multiple times. If all assignments are with δ -delay, only the last assignment takes effect.

```
signal a, b, c, d, y: std_logic;
...
process(a, b, c, d)
begin
  y <= a or c;
  y <= a and b;
  y <= c and d;
end process;
```



```
signal a, b, c, d, y: std_logic;
...
process(a, b, c, d)
begin
  y <= c and d;
end process;
```

- Although this segment is easy to understand, multiple assignment may introduce subtle mistakes in a more complex code and make synthesis very difficult.

注意：只会延时一个delta，VHDL中执行和延时是分开的两个进程，执行无延时在进程内的信号是顺序处理，并行赋值的

109

- Unless there is a compelling reason, it is a good idea to avoid assign a signal multiple times in a process. The **only exception** is the assignment of a default value in the if and case statement.
- The result will be very different if the multiple assignments are the concurrent signal assignment statements

```
signal a, b, c, d, y: std_logic;
...
-- the statements are not inside a process
y <= a or c;
y <= a and b;
y <= c and d;
```

- The above code is syntactically correct.
- However, the design is incorrect because of the potential output conflict.

110

4.2.2 Variable assignment statement

- Variables can be declared and used inside a process statement.
- Variable assignment statement:
variable-object := expression;
- Variable assignment is immediate.

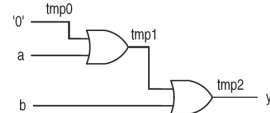
```
Signal a, b, y : std_logic;
.....
process (a, b) is
  variable tmp : std_logic;
  begin
    tmp := '0';
    tmp := tmp or a;
    tmp := tmp or b;
    y <= tmp;
  end process;
```

Although the behavior of a variable is easy to understand, mapping it to hardware is difficult. **Not recommended.**

111

□ To realize the previous process in hardware, we have to rename the variables.

```
process (a, b) is
  variable tmp0, tmp1, tmp2 : std_logic;
  begin
    tmp0 := '0';
    tmp1 := tmp0 or a;
    tmp2 := tmp1 or b;
    y <= tmp2;
  end process;
```



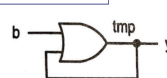
For the lack of clear hardware mapping, we should try to use signal in code in general and resort to variable only for the characteristic that cannot be described by signals.

□ For comparison purpose

```
Signal a,b,y,tmp : std_logic;
process (a, b, tmp) is
begin
  tmp <= '0';
  tmp <= tmp or a;
  tmp <= tmp or b;
  y <= tmp;
end process;
```

```
Signal a,b,y,tmp : std_logic;
process (a, b, tmp) is
begin
  tmp <= tmp or b;
  y <= tmp;
end process;
```

Not recommended.



112

4.2.3 If statement

- An if statement selects a sequence of statements for execution based on the value of a condition.

- General form of an if statement

```
if condition then
  sequential-statements
{ elsif condition then
  sequential-statements }
[ else
  sequential-statements ]
end if;
```

```
if SUM <= 100 then
  SUM := SUM + 100;
end if;
```

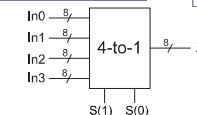
```
if NICKLE_IN then
  DEPOSITED <= T5;
elsif DIME_IN then
  DEPOSITED <= T10;
elsif QUARTER_IN then
  DEPOSITED <= T25;
else DEPOSITED <= TError;
end if;
```

Note the absence of the letter 'e' in elsif !

113

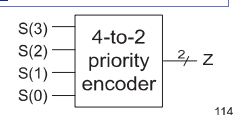
4-to-1 multiplexer based on an if statement

```
architecture if_arch of mux4 is
begin
  process(In0, In1, In2, In3, S)
  begin
    if (S = "00") then
      Z <= In0;
    elsif (S = "01") then
      Z <= In1;
    elsif (S = "10") then
      Z <= In2;
    else Z <= In3;
    end if;
  end process;
end if_arch;
```



4-to-2 priority encoder based on an if statement

```
architecture if_arch of pr_encoder is
begin
  process(S)
  begin
    if (S(3) = '1') then
      Z <= "11";
    elsif (S(2) = '1') then
      Z <= "10";
    elsif (S(1) = '1') then
      Z <= "01";
    else Z <= "00";
    end if;
  end process;
end if_arch;
```



114

Comparison to a conditional assignment statement

- An if statement is somewhat like a concurrent conditional signal assignment statement.

```
sig <= value_exp1 when boolean_exp1 else
value_exp2 when boolean_exp2 else
value_exp3 when boolean_exp3 else
...
value_expn;
```

115

Process(...)

```
begin
  if boolean_exp1 then sig <= value_exp1;
  elsif boolean_exp2 then sig <= value_exp2;
  elsif boolean_exp3 then sig <= value_exp3;
  ...
  else sig <= value_expn;
  end if;
end process;
```

- The above two statements are equivalent.
- In general, equivalent only if each branch of the if statement consists of a single assignment of the same signal.

116

- An if statement allows for arbitrary nesting of if statement.

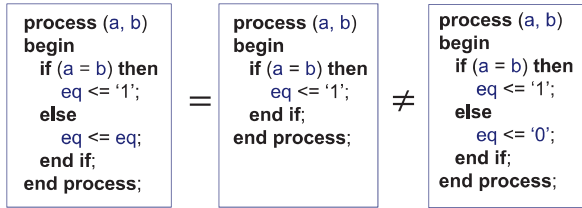
```
process (CTRL1, CTRL2)
begin
  if CTRL1 = '1' then
    if CTRL2 = '0' then
      MUX_OUT <= "0010";
    else
      MUX_OUT <= "0001";
    end if;
  else
    if CTRL2 = '0' then
      MUX_OUT <= "1000";
    else
      MUX_OUT <= "0100";
    end if;
  end if;
end process;
```

Write the equivalent of conditional signal assignment

117

Incomplete if branch

- In VHDL, only the then branch is mandatory and the other branches can be omitted.

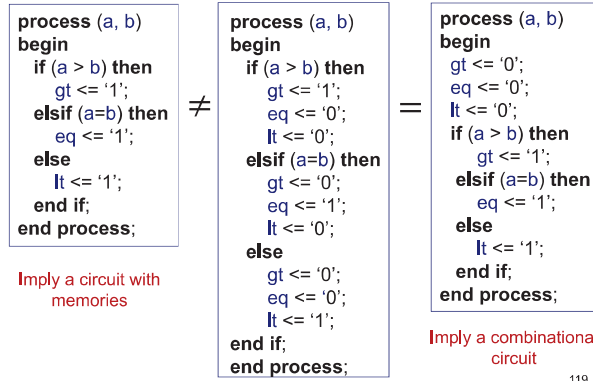


Implies a circuit with a closed feedback loop, which constitutes memory

Implies a combinational circuit

118

Incomplete signal assignment



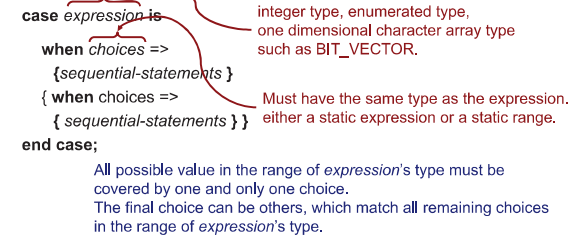
Implies a circuit with memories

Implies a combinational circuit

119

4.2.4 Case statement

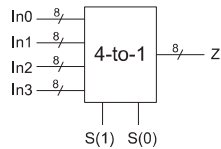
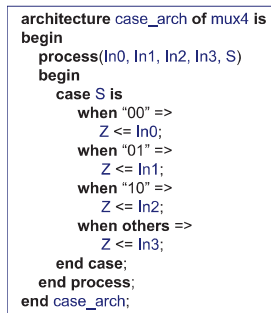
- A case statement is a multiway branch based on the value of a control expression
- General form of a case statement



Choices之间需要互斥且包含所有的情况

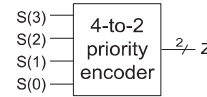
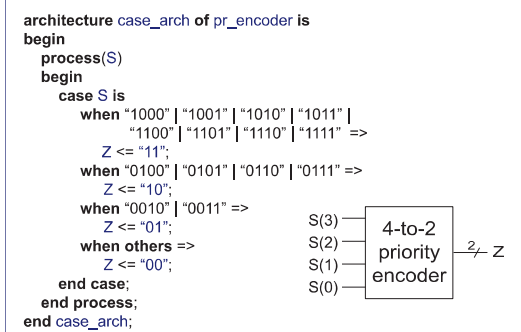
122

4-to-1 multiplexer based on a case statement

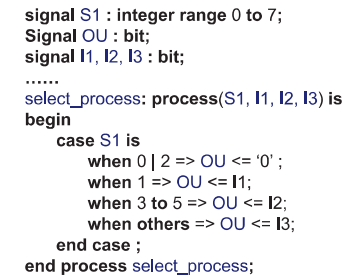


123

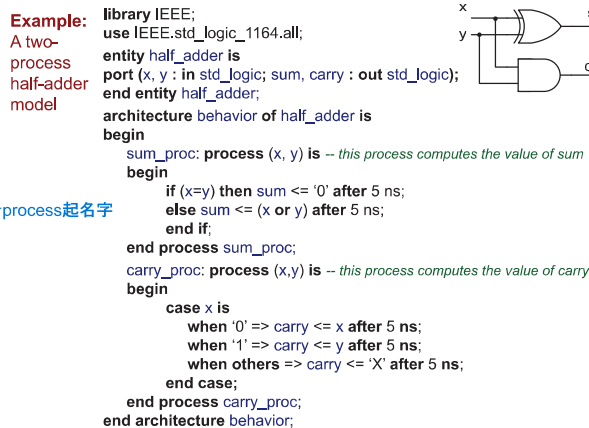
4-to-2 priority encoder based on a case statement



124



125



126

4.2.5 NULL statement

- The statement `null;` is a sequential statement that does not cause any action to take place;
- variable `SEL` : integer range 0 to 31;
- variable `V` : integer range 0 to 31;
- case `SEL` is
- when 0 to 15 => `V := SEL;`
- when others => `null;`
- end case;

128